Assignment 3: Concurrent Web Proxy

Due: April 18th, 11:50PM

1 Introduction

In this assignment, you are required to upgrade your sequential Basic Web Proxy from Assignment 2 so that it can handle multiple clients concurrently using threads or processes. That is, if your proxy is waiting for some event on behalf of one connection, it should be able to handle events for other connections as well.

This will give you experience with concurrency and synchronization, which are crucial computer systems concepts. You will need to implement two versions of the concurrent web proxy: a **threaded version** and a **multi-process version**.

2 Handout Instruction

There is no handout for this assignment, as the code written for the sequential Basic Web Proxy will serve as the code base for the Concurrent Web Proxy. You can use your own implementation of the Web Proxy or the *proxy.c* (a sample solution to Basic Web Proxy in Resources section on Canvas) as your code base. You will have to reorganize the code base and write additional code to make it work for Concurrent Proxy while maintaining the same project directory structure as in Assignment 2.

3 Concurrent Proxy Server Implementation

Real proxies do not process requests sequentially. They handle multiple requests concurrently. Once you have a working sequential logging proxy (from Assignment 2), you should modify it to handle multiple requests concurrently. There are two approaches to achieve concurrency;

1) **Process based concurrency** using file locking for synchronization:
   - Create a new process to deal with each new connection request that arrives.
2) **Thread based** using mutexes for synchronization:
   - Create a new thread to deal with each new connection request that arrives.

You are required to implement **both versions.** Additionally, you need to ensure the following:

- The proxy should **log all client requests** to a shared log file `proxy.log`.
- Access to shared resources like the log file **must be synchronized** to prevent corruption.

(For example, multiple processes/threads may access the log file concurrently. Without proper synchronization, multiple log events might mix up.)

- Your **multithreaded version must be thread-safe**. Protect calls to thread-unsafe functions such as `gethostbyaddr` and `gethostbyname` to ensure they operate safely in a multithreaded environment. (A piece of code is thread-safe if it only manipulates shared data structures in a manner that guarantees safe execution by multiple threads at the same time.)
- The concurrent proxy **must maintain all the functionalities and features** from Assignment 2.

3.1 Selecting the Concurrent Version

Your proxy should support both process-based and thread-based concurrency. To specify which version to run, use a command-line option:

- Multi-process version (default behavior if no option is given)

  `./proxy <port-number>`

- Thread-based version (use `-t` or `--thread`):

  `./proxy -t <port-number>` or `./proxy --thread <port-number>`

- (Optional) Multi-process version with `-p` or `--process`:
  - You may implement `-p` or `--process` as an explicit option for running the multi-process version.
  - If implemented, both of the following commands should behave the same:

    `./proxy <port-number>`

    `./proxy -p <port-number>`

- If an invalid option is provided, the program should print usage instructions and exit.

Implementation Requirements:

- Parse command-line arguments to determine which version to run.
- Implement the default multi-process version when no option is specified.
- When the `-t` or `--thread` flag is used, execute the thread-based version.
- Ensure proper error handling for invalid or missing arguments.

4 Logging Requirements

4.1 Logging Event Time

- The proxy should log each request when the full response is received from the destination server.
- This ensures that the response size is included in the log.

4.2 Handling Different Response Status Codes

- The proxy should log all responses, regardless of whether the HTTP status code is 200 OK, 404 Not Found, 500 Internal Server Error, etc.
- However, if the proxy does not receive any response from the server (e.g., timeout or connection failure), the request should not be logged.

4.3 Impact of Advanced Features on Logging

4.3.1 HTTPS Tunneling (CONNECT method)

- Since HTTPS traffic is encrypted, the proxy **cannot log request details or response sizes**.
- Instead, log only **metadata**:

  [Timestamp] CONNECT from [Client IP] to [Target Host]:[Port], Data Transferred: [Bytes Sent] / [Bytes Received]
- The log entry should be recorded **when the tunnel is closed**, capturing total bytes forwarded.

4.3.2 Persistent Connections (Connection: keep-alive)

- Each individual HTTP request-response pair should be logged separately, even if using the same client-proxy connection.
- Log the connection closure separately if it times out or is explicitly closed by either side.

5 Additional Proxy Features

In addition to making the proxy concurrent you are required to implement at least one additional feature to complete this assignment. You can choose between:

- HTTPS Tunneling (CONNECT method)
- Persistent Connections (Connection: keep-alive)

These advanced features provide real-world proxy functionality and require deeper understanding of network protocols.

5.1 HTTPS Tunneling (CONNECT Method)

Most modern websites use HTTPS, which encrypts traffic between the client (browser) and the web server. A basic proxy cannot inspect or modify HTTPS traffic, but it can still support HTTPS connections using the CONNECT method.

How HTTPS Tunneling Works

When a browser wants to establish an HTTPS connection through a proxy, it sends a CONNECT request instead of a regular HTTP request. For example:

```
CONNECT www.example.com:443 HTTP/1.1
Host: www.example.com:443
```

- The proxy should parse the target hostname and port from the request.
- Instead of forwarding an HTTP request, the proxy establishes a TCP connection with the destination server and responds with:

  ```
  HTTP/1.1 200 Connection Established
  ```

- After this, the proxy blindly relays data between the client and the server without interpreting it.

Implementation Requirements

- Detect CONNECT requests and extract the destination hostname and port.
- Establish a TCP connection to the requested server.
- Relay encrypted data in both directions between the client and the server until either side closes the connection.
- Ensure proper socket closure when either side terminates the connection.
  - If the browser closes its connection, the proxy must close the corresponding proxy-server connection.
  - If the server closes its connection, the proxy must close the corresponding browser-proxy connection.
  - If an error occurs or the tunnel is idle for more than 60 seconds, the proxy should gracefully close both connections to free resources. (You can make the idle timeout value configurable if you want more flexibility.)

5.2 Persistent Connections (Connection: keep-alive)

By default, HTTP/1.1 uses persistent connections, meaning the same connection can be reused for multiple requests to the same destination server.

How Persistent Connections Work

- When a browser connects to your proxy, it may send multiple requests over the same connection if Connection: keep-alive is set.
- If your proxy establishes a connection to a web server, that connection can also be reused for multiple requests to the same server.
- This avoids the overhead of creating new TCP connections for each request.

Proxy Behavior with Persistent Connections

1. The browser-proxy connection remains open for multiple requests, as long as the browser doesn't close it.
2. The proxy-server connection remains open for multiple requests to the same destination server.
3. Each browser-proxy connection must have its own backend (proxy-server) connection to ensure correct request-response mapping.
    - The proxy should NOT share a backend connection between multiple browser-proxy connections, because HTTP/1.1 does not provide a way to match responses correctly in this case.
    - If a browser opens two persistent connections to the proxy and sends requests to the same server, the proxy must open two separate backend connections to that server.
4. The proxy simply forwards requests and responses over these existing connections, ensuring that each response is sent back to the browser connection that made the original request.
5. If the browser requests a different server, a new connection must be created to that destination.
6. If either the browser or server closes the connection, the proxy may need to close the corresponding connection(s) on the other side (see below).

When to Close Persistent Connections

Your proxy should close a persistent connection if:

1. The client (browser) closes the connection
    - The proxy must close all backend connections associated with that browser connection.
2. A backend server closes its connection, but other backend connections exist.
    - The proxy should only close that specific backend connection.

- The browser connection should remain open if there are other active backend connections.

3. All backend connections for a browser-proxy connection are closed
   - The proxy should then close the browser connection, since no further requests can be processed.
4. The client (browser) sends Connection: close
   - The proxy should close both the browser-proxy connection and all associated backend connections.
5. The connection is idle for too long (e.g., 60-240 seconds without activity)
   - If neither the browser nor any backend server sends data within a timeout period, the proxy should close both the browser connection and any remaining backend connections.

Handling Multiple Destination Servers in Persistent Connections

- When a browser establishes a persistent connection with the proxy, it may send multiple requests to different web servers over the same connection.
- The proxy must determine for each request:
  1. If a backend connection to the target web server already exists, reuse it.
  2. If no existing backend connection exists, open a new TCP connection to the requested web server.
- This means that the proxy must track multiple active backend connections and correctly map each request to the appropriate server connection.
- The proxy should not mix requests between servers—each backend connection should only handle requests for its corresponding web server.

Example Scenario

A browser opens a single persistent connection to the proxy and sends these requests:

```
GET http://www.example.com/page1.html HTTP/1.1
```

```
GET http://www.another-site.com/home.html HTTP/1.1
```

- The proxy must forward each request to the correct web server.
- It should reuse a single proxy-server connection for www.example.com and a different proxy-server connection for www.another-site.com.

6 Testing your Proxy

There is no script to check your implementation. You will need to come up with your own tests to debug your code and determine when your implementation is correct.

Tip:

- Use web testing tools like curl or HTTPie to simulate multiple clients making concurrent requests to your proxy.

  ```
  curl -x http://localhost:15213 http://www.example.com/
  http --proxy=http://localhost:15213 http://www.example.com/
  ```

- Temporarily add a 2-second delay before logging to help observe concurrency behavior.

  ```
  sleep(2);        // Debugging only, remove after testing!
  ```

- Remember to remove the delay before submitting your assignment!

7 Submission Instructions

Your final score is based on the following grading breakdown:

Feature	Points
Basic Concurrent Proxy (Multi-Process & Multi-Threaded)	12
HTTPS Tunneling (CONNECT)	3
Persistent Connections (keep-alive)	5
Total Possible Score	20

- Submit your complete project directory on Canvas.
- Create a gzipped tar ball that includes all your source code.
- The filename for the tarball must be `username-concurrent-proxy.tar.gz` where `username` is your NETID.
- All your files in the tarball **must** be in a directory named `username-concurrent-proxy`.
- Include a README file explaining how to compile/test both versions of the proxy implementations.
- Submission is due on the date mentioned on Canvas.